7. Implementation

This section describes the TuneBoxd features that are implemented in our code as of the end of Sprint 3. Sprint 2 delivered user management (registration, login, logout, sessions and profiles) together with following other members. Sprint 3 adds music discovery through the MusicBrainz API, album pages, reviews with ratings, a listening diary with privacy controls, and comments on reviews. All user-generated data is stored in our shared MySQL database on Amazon RDS. Features that appear in our requirements but are not yet implemented are listed in Section 7.2.7.

7.1 Tech Stack

TuneBoxd is a full-stack web application built with **React and Vite** for the frontend, **Node.js and Express** for the backend, and a **MySQL** database hosted on **Amazon RDS**. Music information comes from the public **MusicBrainz** API, and album artwork comes from the **Cover Art Archive**. This stack keeps the user interface, the application logic and the stored data clearly separated, while staying lightweight enough for a semester project. Using JavaScript on both the frontend and the backend also means every team member can work on either side without switching languages.

Table 7.1: Technologies used in TuneBoxd

Technology	Role in TuneBoxd	Why we chose it
React 19	Builds the user interface from reusable components (forms, navigation, review cards, diary rows).	Component-based structure lets us reuse pieces such as the star rating and review card across many pages, and the UI updates automatically when the login state changes.
React Router 7	Client-side page routing (e.g. `/album/:mbid`, `/diary`, `/u/:username`) and protected routes.	Standard routing library for React; supports redirecting logged-out users to the login page.
Vite 6	Development server and production build tool for the frontend.	Fast startup and instant page updates during development with very little configuration. Its proxy forwards `/api` requests to Express, so no CORS setup is needed.
Node.js (20.12 or later)	Runs the backend server.	Lets the backend be written in JavaScript, the same language as the frontend.
Express 5	Defines the REST API routes, middleware and central error handling.	Simple, well-documented way to organize routes and authentication checks.
MySQL 8 on Amazon RDS	Stores users, follows, artists, albums, songs, reviews, ratings, comments and diary entries.	A relational database fits our data, which has many relationships (users to reviews, reviews to comments, and so on). Hosting on RDS gives the whole team one shared database.
mysql2	Node.js driver; maintains a connection pool and runs parameterized (prepared) queries.	Supports promises and prepared statements, which protect against SQL injection.

Technology	Role in TuneBoxd	Why we chose it
MySQL Workbench	Used by the team to connect to RDS, inspect tables and run verification queries.	Graphical tool for checking that the application saved the correct data.
express-session	Creates a server-side session after login and sends an HTTP-only session cookie.	Straightforward session-based authentication that works well with a same-origin React app.
bcryptjs	Hashes passwords (10 salt rounds) before storing them and compares hashes at login.	Passwords are never stored as plain text, and bcrypt is slow by design, which makes guessing attacks harder.
MusicBrainz API	Provides artist, album (release group) and song (recording) metadata, search, release dates, tracklists and genres.	Free, open music database that does not require an API key, with stable identifiers (MBIDs).
Cover Art Archive	Provides album cover images linked to MusicBrainz IDs.	Companion service to MusicBrainz, so covers can be found using the same IDs.
node:test (built into Node.js)	Runs our automated unit and end-to-end API tests.	No extra testing library needs to be installed.

How the pieces work together. The user opens the React app in a browser. During development, Vite serves the app on `http://localhost:5173` and forwards every request that starts with `/api` to the Express server on port 4000, so the browser only ever talks to one address and the session cookie works without extra configuration. Express checks the session, validates the input and then either queries MySQL on RDS through the mysql2 connection pool or calls MusicBrainz and Cover Art Archive. The browser never calls the MusicBrainz API directly; only album cover images are loaded straight from Cover Art Archive.

On the backend, the code follows a layered structure: **routes** map URLs to **controllers**, controllers call **services** that contain the business rules (validation, ownership checks, privacy), and services use **repositories**, which are the only files that contain SQL. This made it easy to add the Sprint 3 features without changing how the Sprint 2 code works.

7.2 Features and Major Use Cases Implemented

7.2.1 Sprint 2 Summary: User Management

Sprint 2 implemented account management, which every Sprint 3 feature depends on. New users register on the Sign Up page with a username, email address, password and password confirmation. The same rules are checked on the frontend for quick feedback and again on the backend so they cannot be bypassed: usernames must be 3–30 letters, numbers, underscores or periods; the email must have a valid format; passwords must be 8–72 characters and must match; and usernames and email addresses must be unique. Passwords are hashed with bcryptjs before the row is inserted into the `users` table, and new accounts always receive the regular `user` role.

Users can view their profile and edit their display name, bio, username and email on the Edit Profile page; the backend re-validates and re-checks uniqueness before saving. Users can also

browse other members on the Community page and follow or unfollow them, which is stored in the `follows` table. Profile picture uploads are only partially implemented: the interface exists, but the backend currently rejects uploads because the `users.avatar_url` column (VARCHAR(500)) is too small to store the image.

7.2.2 Log In and Log Out

What it does. Registered users can sign in with either their username or their email address, stay signed in while they move between pages or refresh, and sign out securely.

User interaction. The user enters their username or email and password on the Login page. After a successful login they are taken back to the page they were trying to use (for example, an album they wanted to review) or to their profile. The top bar then shows Profile and Logout instead of Login and Sign Up, and the sidebar shows My Diary. Clicking Logout returns the user to the Login page with the message "You have been logged out."

Frontend–backend communication. The Login page sends `POST /api/auth/login` with `{ identifier, password }`. When the app first loads, React calls `GET /api/me`; a valid session cookie returns the current user, which restores the logged-in state after a refresh. Logout sends `POST /api/auth/logout`.

Backend operations. The server looks up the user (by email if the identifier contains "@", otherwise by username), compares the password with the stored bcrypt hash and checks that `users.account_status` is `active`. It then regenerates the session to prevent session fixation and stores only the user ID in the session. Logout destroys the session on the server and clears the `tuneboxd.sid` cookie.

Database. Reads the `users` table. Sessions themselves are held in the server's memory (express-session's default store), not in MySQL.

Authentication and authorization. The session cookie is HTTP-only (JavaScript cannot read it), uses SameSite=Lax and expires after 7 days. A `requireAuth` middleware protects every route that needs a logged-in user; it re-reads the user from MySQL on each request, so a suspended or deleted account immediately loses access. On the frontend, protected pages (Profile, Edit Profile and My Diary) redirect logged-out visitors to the Login page with the message "Please log in to continue."

Validation and error handling. Missing fields return "All fields are required." A wrong password or unknown account returns the same message, "Incorrect username/email or password.", so the response does not reveal which accounts exist. Suspended accounts receive "This account has been suspended."

Limitation: because sessions are stored in the backend's memory, restarting the backend logs everyone out (accounts and data are unaffected). A persistent session store would need a `sessions` table that is not yet in our schema.

7.2.3 Use Case 1: View Album Pages and Music Discovery

What it does. Anyone, including visitors who are not logged in, can search for albums, artists and songs, open an album page with its cover, artist, release date, genres and full tracklist, and read the reviews and average rating that TuneBoxd users have left. Artist and song pages are also implemented.

User interaction. The user types into the search bar at the top of every page (or opens Search in the sidebar). The Search page shows tabs for Albums, Artists and Songs, with album covers in a grid and Previous/Next paging. Clicking a result opens `/album/:mbid`, `/artist/:mbid` or `/song/:mbid`. Track titles on an album page link to the song page, and artist names link to the artist page, which lists the artist's albums and EPs.

Frontend–backend communication. The Search page calls `GET /api/music/search?query=...&type=album|artist|song`. An album page makes two requests at the same time: `GET /api/music/albums/:mbid` for the music information and `GET /api/music/albums/:mbid/reviews` for TuneBoxd's reviews and rating statistics.

Backend operations. The backend validates the MusicBrainz ID, requests the album (a MusicBrainz *release group*) and then the tracklist of one representative edition (the earliest official release with an exact date), and fetches the cover from Cover Art Archive. The reviews endpoint finds the album's local row by its MusicBrainz ID and returns its reviews, newest first, along with the average rating, number of ratings and number of reviews.

Database. Album metadata is not copied into MySQL just because someone viewed it. The reviews endpoint reads `albums`, `reviews`, `ratings` and `users`. If nobody has reviewed an album yet, it has no local row and the page shows "No reviews yet."

Authentication. Browsing and reading reviews are public. The "Write a Review" and "Add to Diary" buttons are visible to everyone, but a logged-out visitor who clicks them is sent to the Login page and returned to the album afterwards.

External APIs. Uses MusicBrainz for all metadata and Cover Art Archive for covers (details in Section 7.6).

Validation and error handling. Empty searches and searches longer than 100 characters are rejected. Searches with no matches show a "No results" message. An invalid ID shows "Album not found" (HTTP 400/404). If MusicBrainz is unavailable or rate-limiting, the page shows "MusicBrainz is temporarily unavailable" with a Try again button. Albums without artwork display a vinyl-style placeholder, and a missing tracklist shows a message instead of breaking the page.

7.2.4 Use Case 2: Write and Edit Reviews

What it does. Logged-in users can write a review with a star rating for an album or a song, edit it later, and delete it. Each user can have one review per album and one per song. Everyone can read reviews.

User interaction. On an album or song page, the user clicks **Write a Review**, chooses a rating from 0.5 to 5 stars in half-star steps (by clicking the left or right half of a star, or with the arrow keys), types the review and clicks **Post Review**. The message "Your review was saved." only appears after the server confirms the save, and the review then appears in the Reviews list with the author, rating, date and text. If the user already has a review, the button reads **Edit Your Review** and opens the form pre-filled. The author also sees Edit and Delete links on their own review card; other users do not. A user's reviews are listed on their profile (Overview and Reviews tabs), and each review has its own page at `/reviews/:id`.

Frontend–backend communication. New reviews are sent with `POST /api/reviews` and the body `{ targetType: "album" | "song", mbid, text, rating }`. Edits use `PATCH /api/reviews/:id` with `{ text, rating }`, and deletion uses `DELETE /api/reviews/:id`. No user ID is ever sent from the browser.

Backend operations. The author is always taken from the session. Before saving, the backend looks up the album or song on MusicBrainz to confirm it exists, then makes sure the matching rows exist in MySQL: it inserts or updates the artist and album (or song) using their MusicBrainz IDs, so many users reviewing the same album share one `albums` row. The review text and the rating are then saved together in a single transaction. On edit, `created_at` is never changed and `updated_at` is set to the current time. Deleting a review also deletes its rating, and its comments are removed automatically by the foreign key's ON DELETE CASCADE rule.

Database. Writes `reviews` (user, album or song, text, timestamps) and `ratings` (score from 0.5 to 5.0), and inserts or updates `artists`, `albums` and `songs`. Our schema stores the score in the separate `ratings` table, so a review is linked to its rating through the same user and album (or song).

Authentication and authorization. Creating, editing and deleting require a session (401 otherwise). Only the author can edit or delete a review; anyone else receives 403 "You can only change your own reviews." This check happens on the server, not only by hiding the buttons.

Validation and error handling. Review text cannot be empty or only whitespace and is limited to 5,000 characters. The rating is required and must be between 0.5 and 5 in half-star steps. A second review of the same item returns 409 with a prompt to edit the existing review. Unknown album or song IDs return 404. Review text is displayed by React, which escapes HTML, so text such as `<script>` appears as plain text instead of running. Reviews hidden by moderation (a report on `review_reports` with status `removed`) and reviews by suspended users are excluded from every list and cannot be edited or commented on.

7.2.5 Use Case 3: Manage Listening Diary

What it does. Logged-in users can record the albums and songs they listen to, including the date and optional notes, edit or delete those entries, and choose whether their diary is public or private. The same album can be logged many times on different days.

User interaction. There are two ways to add an entry. On an album or song page, the user clicks **Add to Diary**, picks the listening date (today by default) and optionally writes notes. On the **My Diary** page, the user clicks **Add Entry**, searches MusicBrainz for an album or song inside the page, selects a result and fills in the same form. My Diary lists entries grouped by month, newest first, with the day, cover, title, artist and notes. Each entry has Edit and Delete links, and deleting asks "Delete this diary entry?" before anything is removed. A Public/Private switch sets the diary's visibility. Other users can see a public diary on the Diary tab of that user's profile.

Frontend–backend communication. `GET /api/diary` returns the user's entries and visibility; `POST /api/diary` creates an entry (`{ targetType, mbid, listenedOn: "YYYY-MM-DD", notes }`); `PATCH /api/diary/:id` and `DELETE /api/diary/:id` edit and delete; `PUT /api/diary/settings` changes visibility; `GET /api/users/:username/diary` returns another user's diary if it is public.

Backend operations. Creating an entry uses the same find-or-create step as reviews to make sure the album or song exists locally, then inserts the entry for the session user. Privacy is checked in the diary service: if a diary is private and the person asking is not its owner, the server returns 403 "This diary is private." A private diary's entry count is also hidden from the profile statistics.

Database. Uses `diary_entries` (owner, album or song, `listened_at`, notes, `created_at`, `updated_at`) and the existing `diary_visibility` column of `user_settings` (a user with no settings row is treated as `public`, the schema default). The date the user listened (`listened_at`) is stored separately from the time the row was created (`created_at`).

Authentication and authorization. All `/api/diary` routes require a session. Users can only edit or delete their own entries (403 otherwise), and private diaries are protected by the server.

Validation and error handling. The date is required, must be a real calendar date, cannot be in the future and cannot be before 1900. Notes are optional and limited to 1,000 characters. A user with no entries sees a friendly empty state with buttons to log a first listen or browse music.

7.2.6 Use Case 4: Post Comments on Reviews

What it does. Users can reply to other users' reviews. Anyone can read comments; logged-in users can post comments and can edit or delete their own.

User interaction. Each review card has a comment button that shows the comment count (for example, "1 comment"). Clicking it opens the comment section below the review, which lists each comment with the author's name, username, date and text, oldest first. Logged-in users see a

text box and a **Post Comment** button; visitors see a "Log in to comment" link. Comment authors see Edit and Delete links (deleting asks for confirmation). If there are no comments, the section says "No comments yet."

Frontend–backend communication. `GET /api/reviews/:id/comments` loads comments; `POST /api/reviews/:id/comments` with `{ text }` posts one; `PATCH /api/comments/:id` and `DELETE /api/comments/:id` edit and delete.

Backend operations. Before saving, the server checks that the review exists and is visible (not removed by moderation). The comment is linked to the review and to the session user. Edits update `updated_at`, and the interface marks edited comments.

Database. Uses the `comments` table, which references `reviews` and `users`. Because `comments.review_id` uses ON DELETE CASCADE, deleting a review deletes its comments, so orphaned comments can never be shown.

Authentication and authorization. Posting needs a session (401 otherwise). Only the comment's author can edit or delete it (403 otherwise).

Validation and error handling. Comments cannot be empty and are limited to 1,000 characters. Commenting on a missing or removed review returns 404. Comment text is rendered safely by React.

7.2.7 Implementation Status

The table below compares the features in our requirements with what is currently implemented in the code.

Table 7.2: Implementation status of TuneBoxd features

Feature	Status	Notes
Register, log in, log out, sessions	Implemented	Sessions are lost when the backend restarts (in-memory store).
View and edit own profile	Partially implemented	Display name, bio, username and email work. Profile picture upload is rejected because `avatar_url` is VARCHAR(500).
Follow / unfollow users	Implemented	No notifications are sent (Use Case 6 mentions notifications).
Search artists, albums and songs	Implemented	MusicBrainz, 20 results per page.
View album, artist and song pages	Implemented	
Rate and review albums and songs; edit and delete reviews	Implemented	A rating is required with every review (R-04 describes the rating as optional).
Listening diary with public/private visibility	Implemented	Visibility is set for the whole diary, not per entry.
Comment on reviews	Implemented	

Feature	Status	Notes
Like reviews	Not implemented	`review_likes` table exists but is unused.
Custom lists and list visibility	Not implemented	Tables exist; sidebar shows "Lists – Soon".
Report inappropriate reviews; admin moderation	Partially implemented	Only the backend hides reviews whose report status is `removed`. There is no report button or administrator screen.
Administrator account management	Not implemented	The `role` column exists but no route checks it.
Listening statistics	Not implemented	
AI recommendations and summaries	Not implemented	Proposed features.

7.3 Demo

GitHub link: *github.com/Vanohra/TuneBoxd*

The screenshots below were captured on October 18, 2026 from the running application (React frontend on localhost:5173 connected to the Express backend and our AWS RDS database). The accounts `tuneboxd_demo_9ead` and `tuneboxd_friend_9ead` were created for this demonstration through the normal Sign Up process.

7.3.1 Home Page

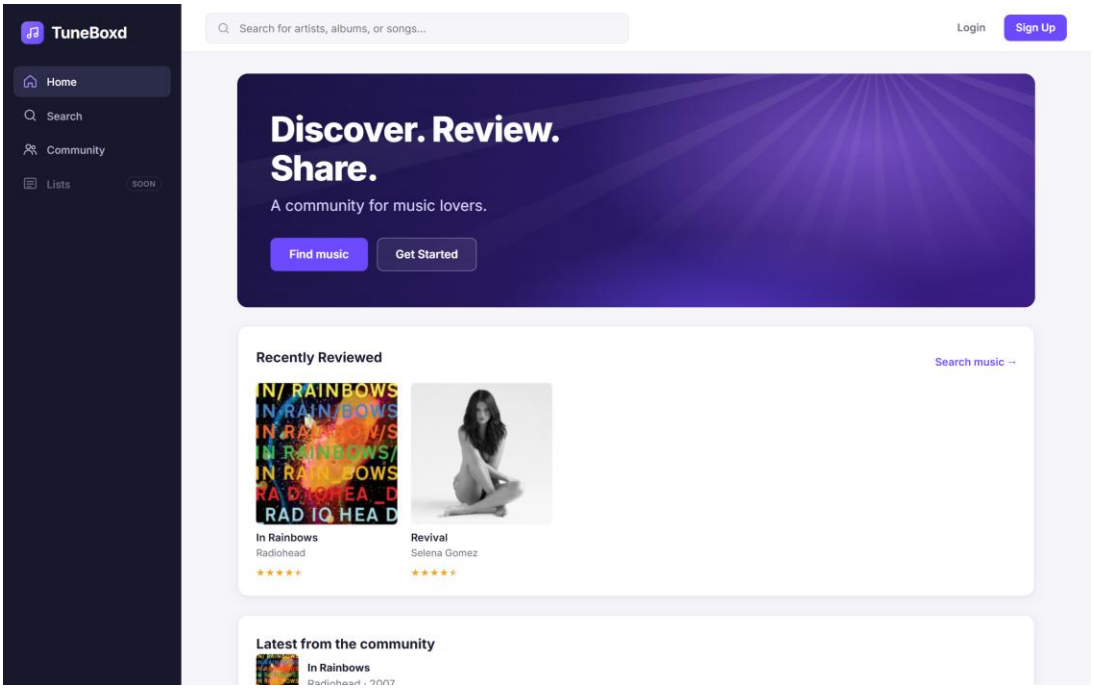


Figure 7.1: Home Page (Logged Out)

The public home page as seen by a visitor who is not logged in. The top bar shows Login and Sign Up, and the sidebar only lists public sections. The Recently Reviewed section is loaded from the `reviews` table in MySQL, and the album covers come from the Cover Art Archive.

7.3.2 Sign Up and Log In

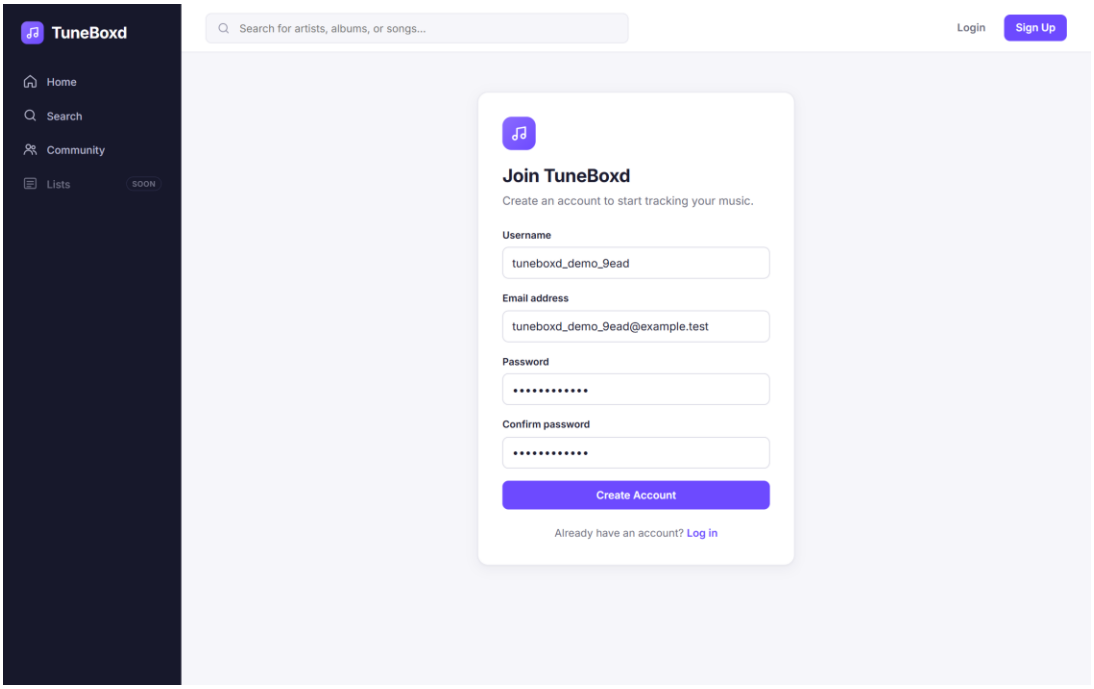


Figure 7.2: Registration Form

A new user fills in the Sign Up form. The password fields are masked. When the form is submitted, the backend validates the fields, hashes the password with bcryptjs and inserts the account into the `users` table.

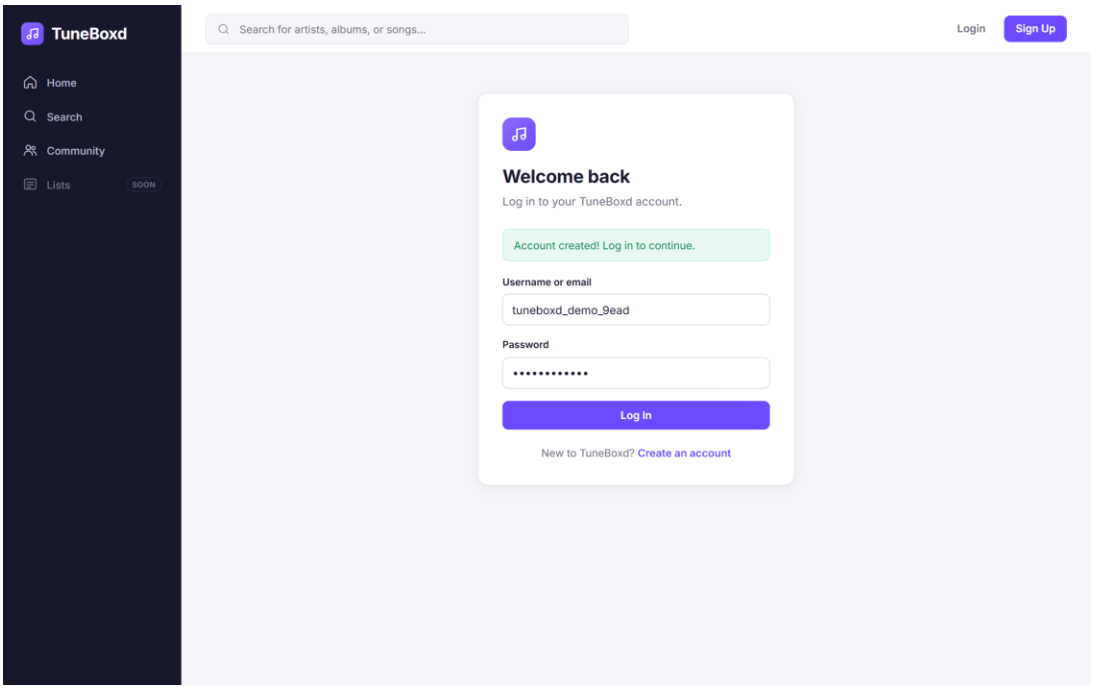


Figure 7.3: Login Page After Registration

After registering, the user is sent to the Login page with the message "Account created! Log in to continue." and the username already filled in. Logging in creates a session (Log In and Log Out use case).

7.3.3 User Profile

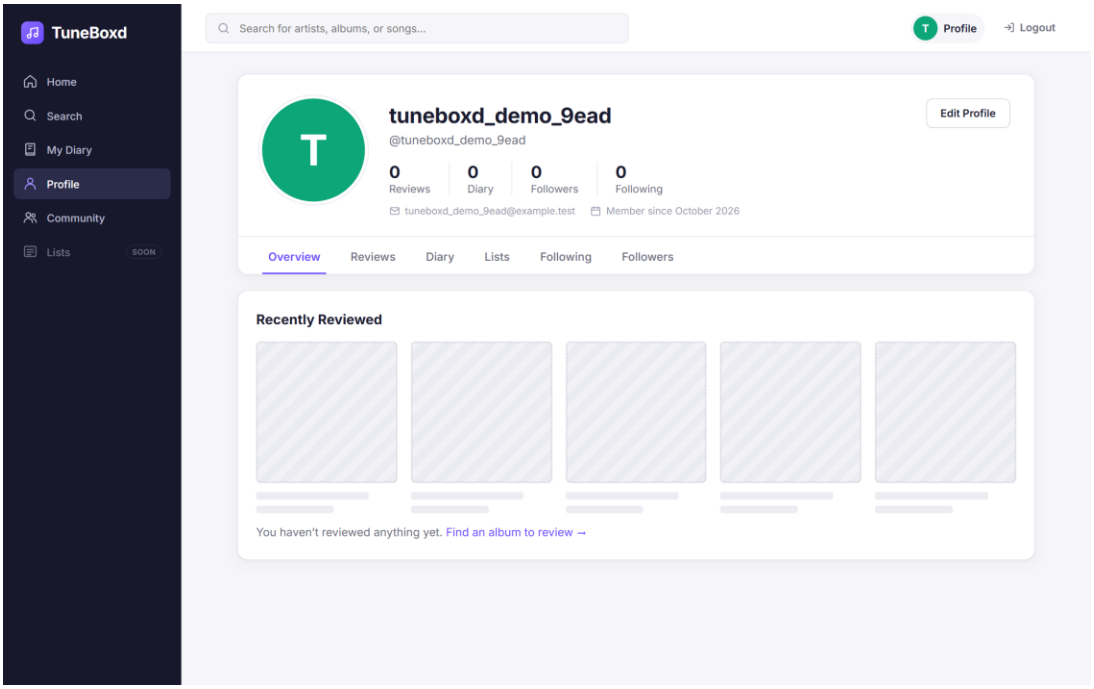


Figure 7.4: Profile of a Newly Registered User

The new user's profile after logging in, retrieved from the `users` table. The statistics show zero reviews, diary entries, followers and following, and the Recently Reviewed section shows an empty state. The top bar now shows Profile and Logout, and the sidebar shows My Diary.

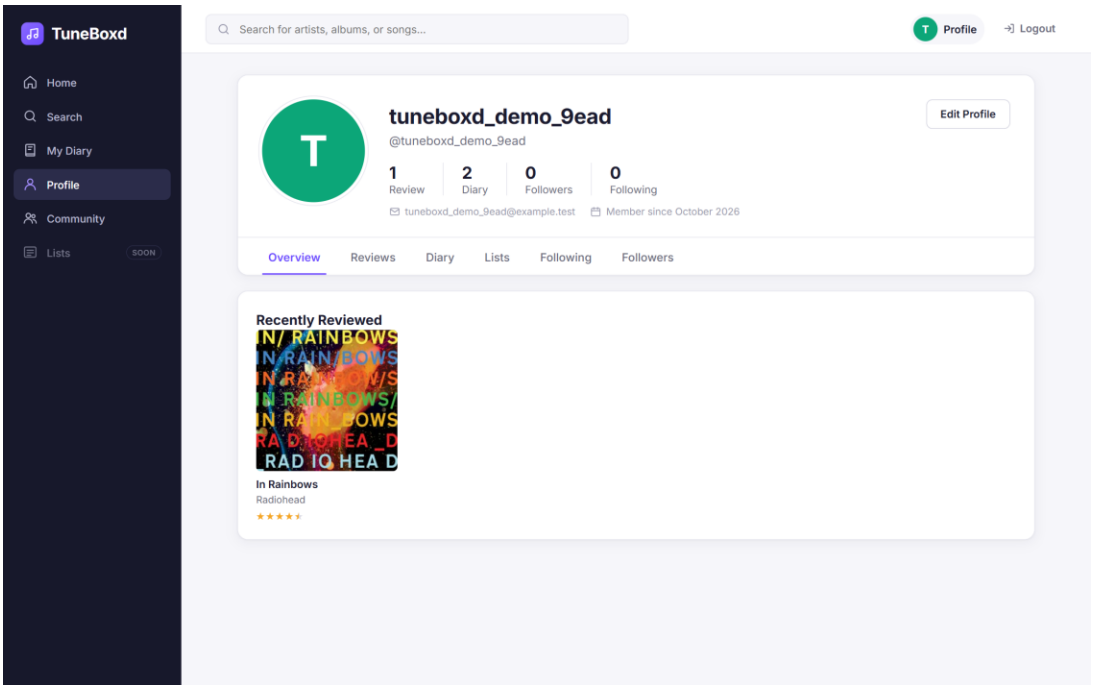


Figure 7.5: Profile After Using the Sprint 3 Features

The same profile after the user wrote a review and logged two listens. The statistics (1 review, 2 diary entries) and the Recently Reviewed cover are retrieved from the `reviews` and `diary_entries` tables. It also shows the application while logged in, before logging out; compare it with Figure 7.13, taken after logging out.

7.3.4 Music Search

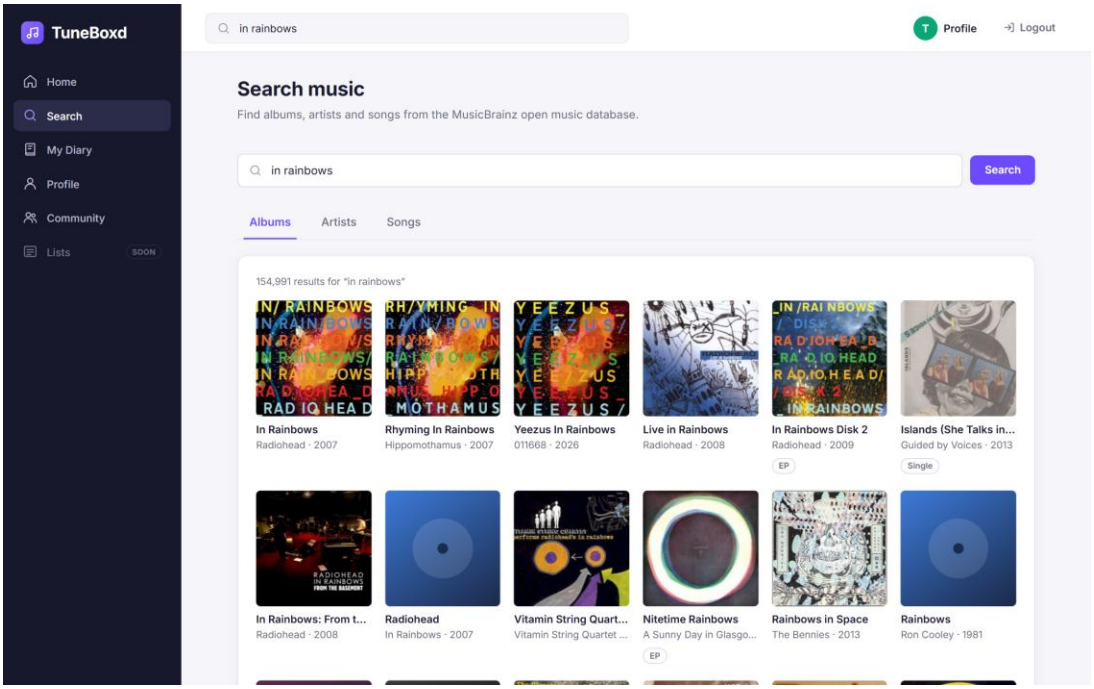


Figure 7.6: Music Search Results

Searching for "in rainbows" from the top bar returns real album results from the MusicBrainz API, retrieved through our Express backend, with covers from the Cover Art Archive. The Albums, Artists and Songs tabs switch the search type (Use Case 1).

7.3.5 Album Details

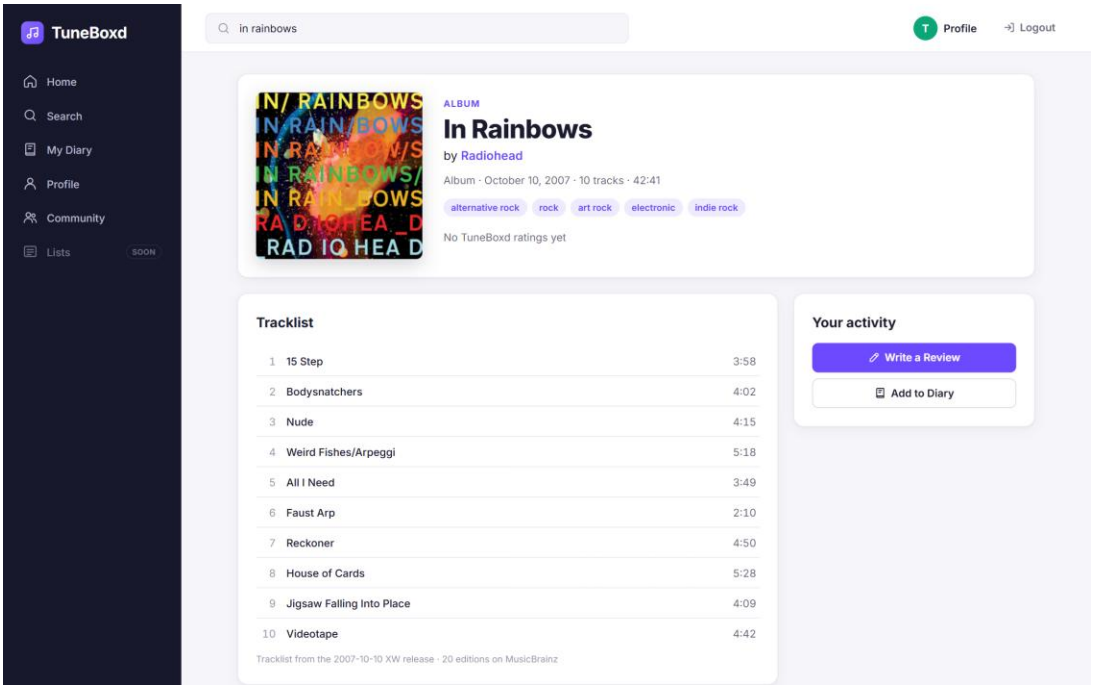


Figure 7.7: Album Details Page

The album page for In Rainbows by Radiohead. The title, artist link, release date, genres, track count, running time, tracklist and artwork are retrieved using the MusicBrainz API and Cover Art Archive. "No TuneBoxd ratings yet" is the result of the reviews query to MySQL, since no one had reviewed this album yet. The Your activity panel provides the Write a Review and Add to Diary actions (Use Case 1).

7.3.6 Writing a Review

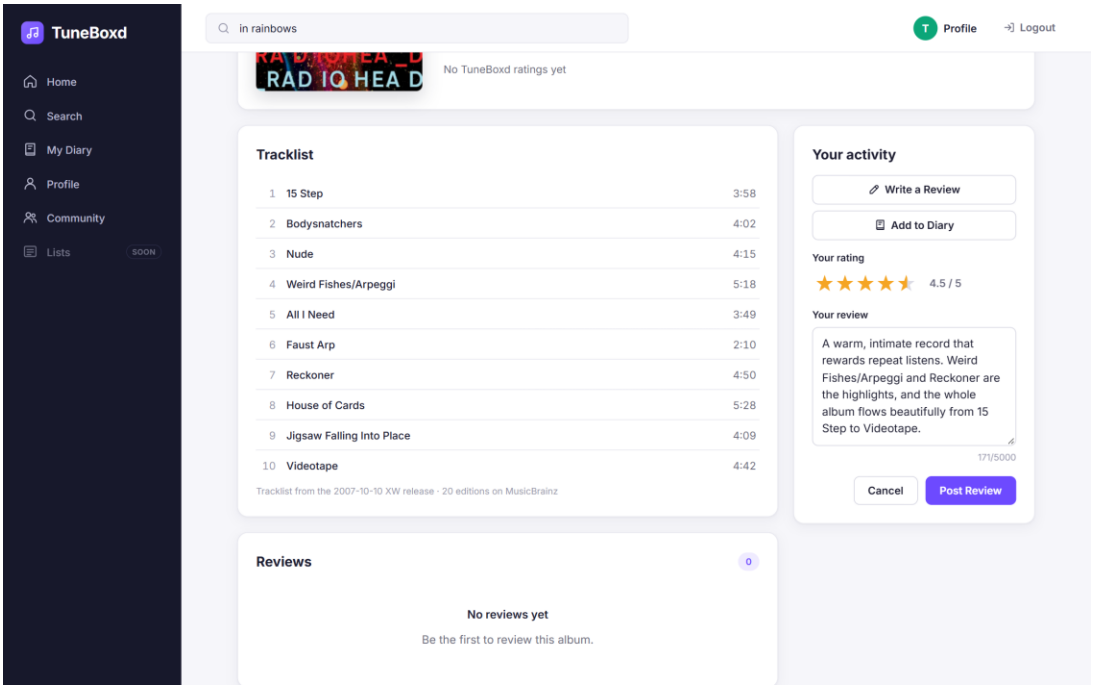


Figure 7.8: Review Submission Interface

After clicking Write a Review, the user selects a 4.5-star rating and writes the review text. The character counter shows the 5,000-character limit (Use Case 2).

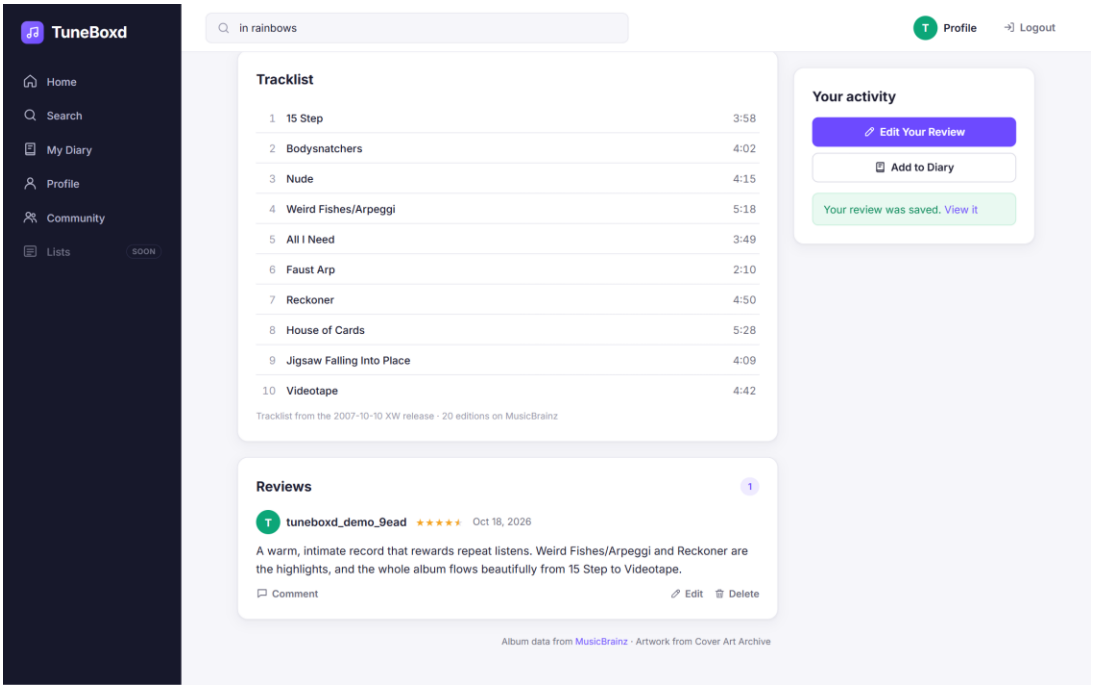


Figure 7.9: Successfully Submitted Review

After the backend confirmed the save, the panel shows "Your review was saved." and the button changes to Edit Your Review. The new review appears in the Reviews list with the author, rating, date and text,

loaded from the `reviews` and `ratings` tables. Edit and Delete appear because this user is the author (Use Case 2).

7.3.7 Listening Diary

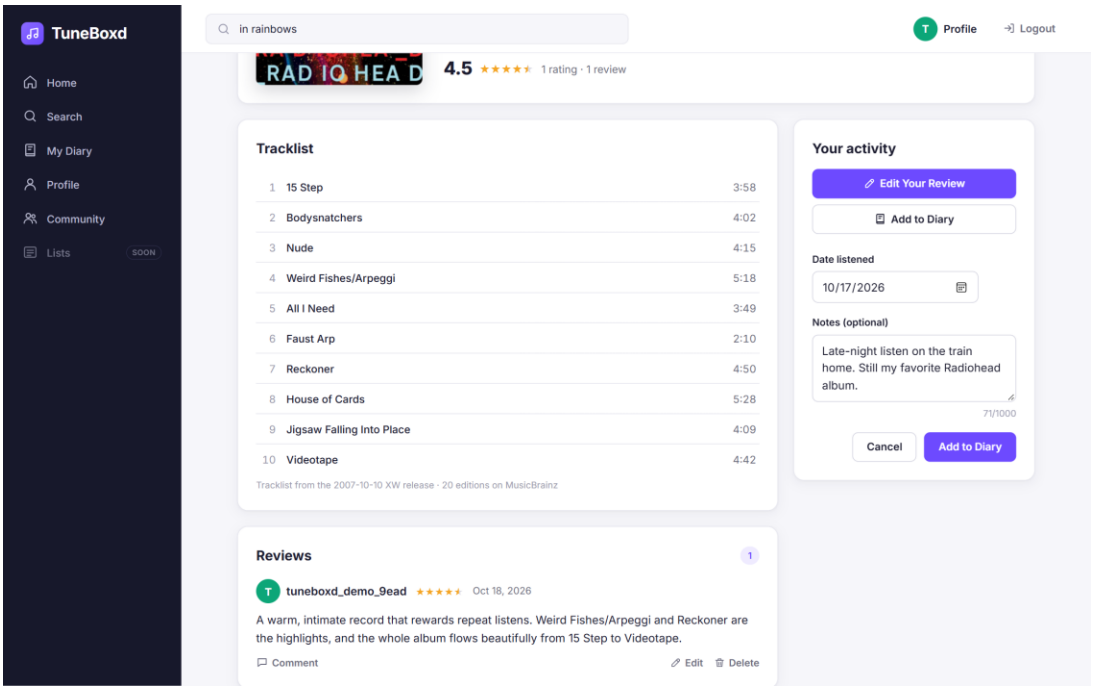


Figure 7.10: Adding a Diary Entry

Clicking Add to Diary on the album page opens a form for the listening date and optional notes. The album's average rating (4.5 from 1 rating) now reflects the review submitted in Figure 7.9 (Use Case 3).

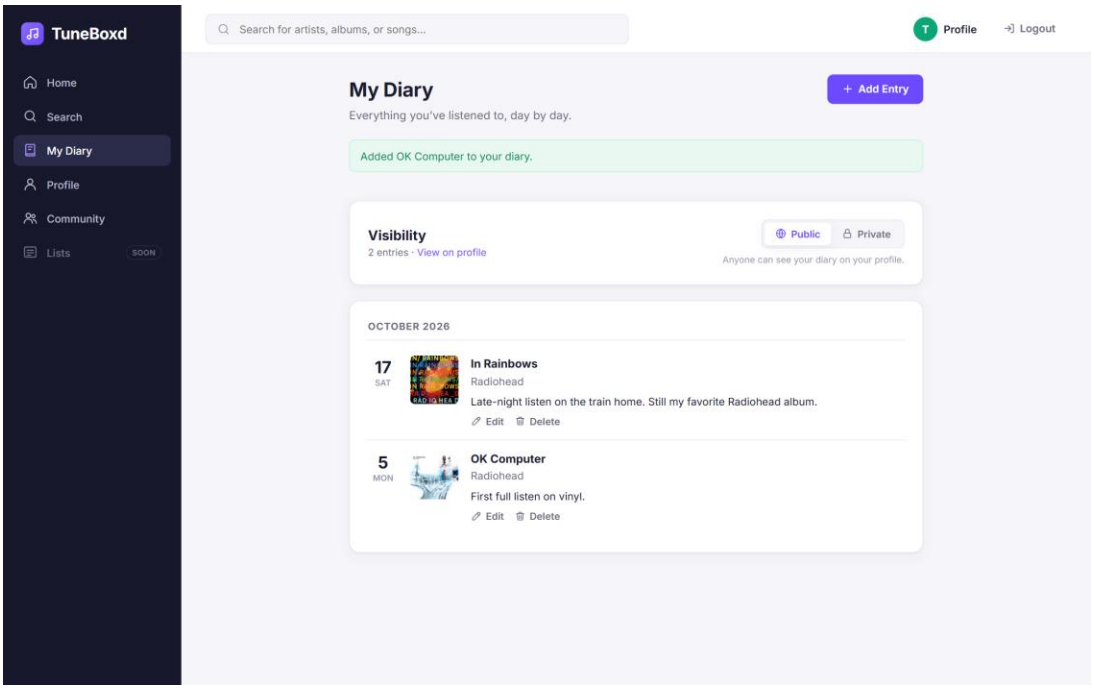


Figure 7.11: My Diary Page

The My Diary page lists the user's entries grouped by month, newest first. The In Rainbows entry was added from the album page, and the OK Computer entry was added with the Add Entry search on this page. Each entry has Edit and Delete links, and the Public/Private switch controls who can see the diary. The data is stored in the `diary_entries` table (Use Case 3).

7.3.8 Commenting on a Review

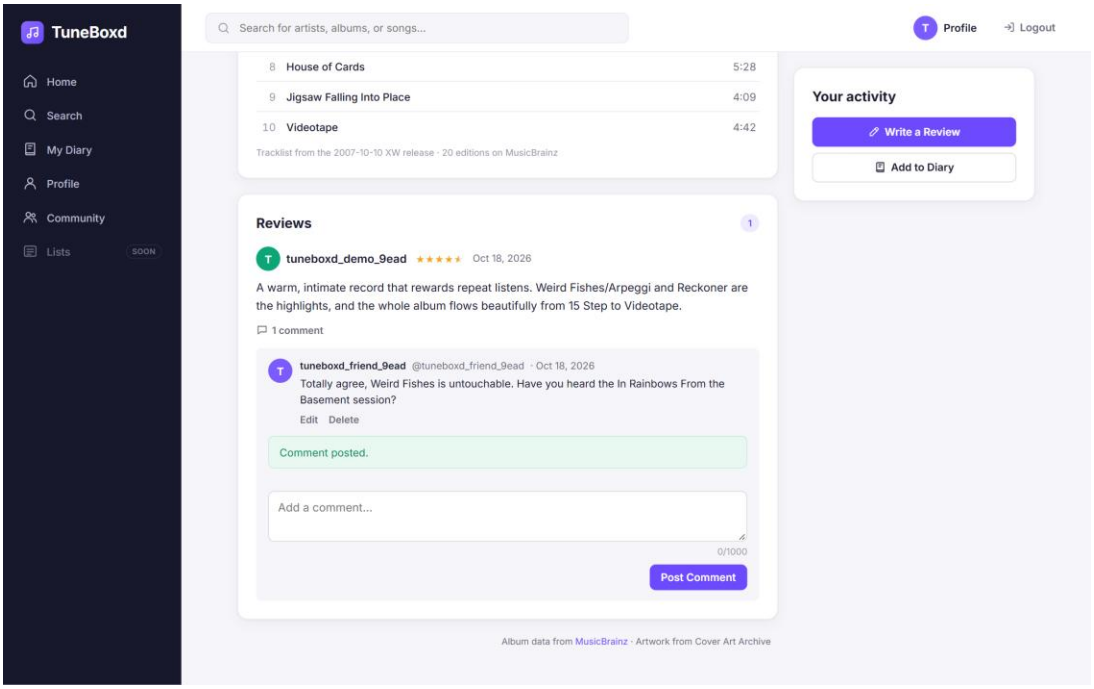


Figure 7.12: Comment Posted on Another User's Review

A second account, tuneboxd_friend_9ead, opened the same album page, expanded the comments under tuneboxd_demo_9ead's review and posted a comment. The comment is saved in the `comments` table, linked to the review and to the commenter, and shows "Comment posted." Only the comment's author sees its Edit and Delete links. Because the review and comment are stored in MySQL, they are visible to any user who opens the page (Use Case 4).

7.3.9 Before and After Logging Out

Figure 7.5 shows the application while logged in: the top bar shows Profile and Logout, and the sidebar includes My Diary and Profile.

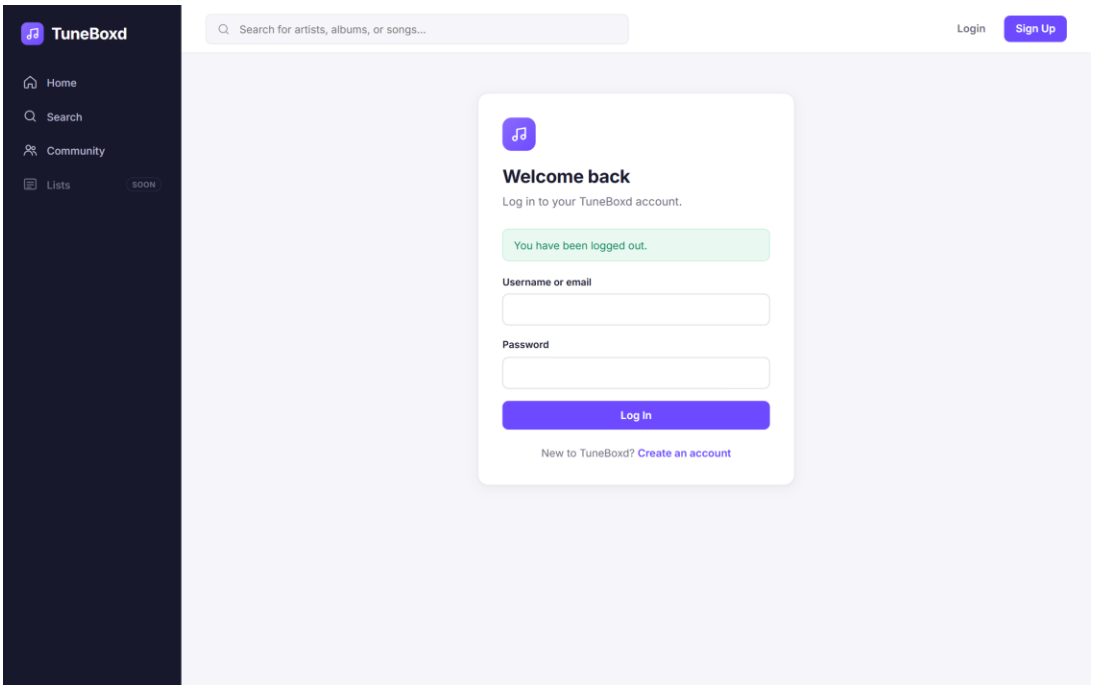


Figure 7.13: After Logging Out

After clicking Logout, the session is destroyed on the server and the user is returned to the Login page with the message "You have been logged out." The top bar shows Login and Sign Up again, and My Diary and Profile disappear from the sidebar (Log In and Log Out use case).

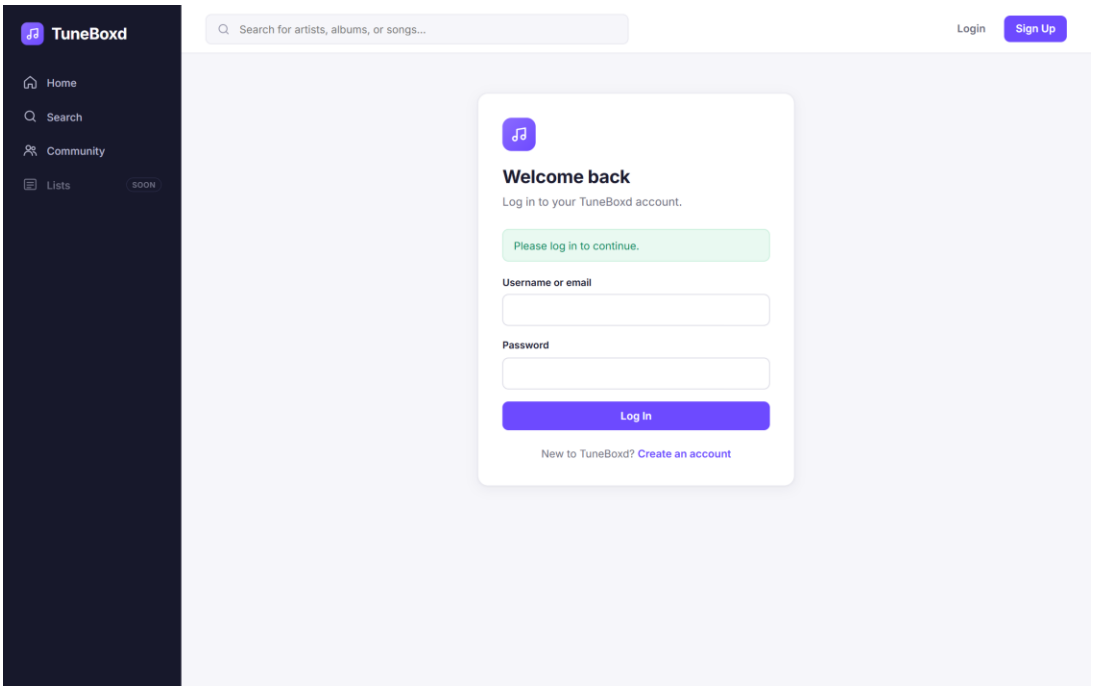


Figure 7.14: Protected Page After Logout

Visiting `/diary` after logging out redirects to the Login page with "Please log in to continue." The backend also rejects `/api/diary` requests without a session (HTTP 401), so the restriction cannot be bypassed by typing the URL.

7.3.10 Community

SCREENSHOT PLACEHOLDER – TO BE CAPTURED BY THE TEAM

The Community page did not change in Sprint 3, so the Community screenshot from the Sprint 2 report is still accurate and can be reused here.

If a new one is preferred: log in, click **Community** in the sidebar, and capture the member list with Follow buttons.

Figure 7.15: Community Page

7.3.11 Database Records in MySQL Workbench

SCREENSHOT PLACEHOLDER – TO BE CAPTURED BY THE TEAM

This screenshot must be captured by a team member, because it requires MySQL Workbench connected to our RDS instance.

1. Open MySQL Workbench and connect to the TuneBoxd RDS connection (database `music_review_app`).

2. Open `database/verify_sprint3.sql` from the repository (File > Open SQL Script) and run it, or run the review query below.

3. Capture the result grid that shows the review by `tuneboxd_demo_9ead` with its rating, and the comment by `tuneboxd_friend_9ead`. A second screenshot of the diary query (two entries) is recommended.

Example query:

```
SELECT r.review_id, u.username, al.title, ra.score,
r.review_text, r.created_at FROM reviews r JOIN users u ON u.user_id =
r.user_id JOIN albums al ON al.album_id = r.album_id JOIN ratings ra ON
ra.user_id = r.user_id AND ra.album_id = r.album_id;
```

Suggested explanation: "The review submitted in Figure 7.9 is stored in the reviews table and linked to the user and album rows; its 4.5-star score is stored in the ratings table."

Figure 7.16: Sprint 3 Records in MySQL Workbench

7.4 API Routes

All routes are served by Express under `/api`. "Yes" means a valid session is required (the server returns 401 without one). "Owner" means a session is required and the server also checks that the user owns the record (403 otherwise). Errors are returned as JSON in the form `{ "error": "message", "fieldErrors": { ... } }`.

Table 7.3: Implemented backend API routes

Method	Endpoint	Purpose	Auth
Authentication			
POST	`/api/auth/register`	Create an account `{ username, email, password, confirmPassword }`	No
POST	`/api/auth/login`	Log in `{ identifier, password }` (username or email)	No
POST	`/api/auth/logout`	Destroy the session and clear the cookie	No
GET	`/api/me`	Current logged-in user	Yes
PATCH	`/api/me`	Update own profile `{ username, email, displayName, bio, avatarUrl }`	Yes
User profiles and following			
GET	`/api/users`	Member list (with follow state when logged in)	No
GET	`/api/users/:username`	Public profile with stats (reviews, diary entries, followers, following)	No
GET	`/api/users/:username/followers`	Users who follow this user	No
GET	`/api/users/:username/following`	Users this user follows	No
GET	`/api/users/:username/reviews`	Reviews written by this user	No
GET	`/api/users/:username/diary`	This user's diary; 403 if private and not the owner	No*
POST	`/api/users/:username/follow`	Follow a user	Yes
DELETE	`/api/users/:username/follow`	Unfollow a user	Yes
Music search and details (MusicBrainz)			

Method	Endpoint	Purpose	Auth
GET	`/api/music/search?query=&type=&page=`	Search albums, artists or songs (`type` = album, artist or song)	No
GET	`/api/music/albums/:mbid`	Album details, tracklist, genres and cover	No
GET	`/api/music/albums/:mbid/reviews`	Album reviews and rating statistics from MySQL	No
GET	`/api/music/artists/:mbid`	Artist details and albums/EPs	No
GET	`/api/music/songs/:mbid`	Song (recording) details	No
GET	`/api/music/songs/:mbid/reviews`	Song reviews and rating statistics	No
Reviews			
GET	`/api/reviews`	Most recent reviews (home page)	No
POST	`/api/reviews`	Create a review `{ targetType, mbid, text, rating }`	Yes
GET	`/api/reviews/:id`	One review	No
PATCH	`/api/reviews/:id`	Edit a review `{ text, rating }`	Owner
DELETE	`/api/reviews/:id`	Delete a review, its rating and its comments	Owner
Comments			
GET	`/api/reviews/:id/comments`	Comments on a review (oldest first)	No
POST	`/api/reviews/:id/comments`	Post a comment `{ text }`	Yes
PATCH	`/api/comments/:id`	Edit a comment `{ text }`	Owner
DELETE	`/api/comments/:id`	Delete a comment	Owner
Listening diary			
GET	`/api/diary`	Own diary entries and visibility setting	Yes
POST	`/api/diary`	Create an entry `{ targetType, mbid, listenedOn, notes }`	Yes
PATCH	`/api/diary/:id`	Edit an entry `{ listenedOn, notes }`	Owner

Method	Endpoint	Purpose	Auth
DELETE	`/api/diary/:id`	Delete an entry	Owner
PUT	`/api/diary/settings`	Set diary visibility `{ visibility: "public" | "private" }`	Yes

* No login is needed, but private diaries are only returned to their owner.

7.5 Database Integration

TuneBoxd uses one shared MySQL 8 database, `music_review_app`, hosted on Amazon RDS. The connection settings (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`) are read from a `.env` file in the repository root, which is excluded from Git so credentials are never committed. The backend opens a connection pool with the mysql2 driver (up to 10 connections) and runs `SELECT 1` at startup; if the database cannot be reached, it prints a clear message such as "Unable to connect to database (ETIMEDOUT)" and stops instead of failing later during login. All queries use `?` placeholders (prepared statements), and only the repository files contain SQL.

Existing versus new tables. The schema in `database/schema.sql` already contained every table the Sprint 3 features need, so **no new tables were created in Sprint 3**. The only schema change was an additive migration, `database/migrations/001_sprint3_diary_updated_at.sql`, which adds a nullable `updated_at` column to `diary_entries` so that edited entries record when they were changed. Existing rows were not modified. The migration can be run more than once safely and was applied to our RDS database on October 18, 2026 with `npm run migrate`.

Table 7.4: Database tables used by each major use case

Use case	Tables read	Tables written	Sprint 3 change
Log in and log out	`users`	`users` (registration, profile edits)	Reused (Sprint 2)
1. View album pages and music discovery	`albums`, `reviews`, `ratings`, `users`	None (metadata comes from MusicBrainz)	Reused
2. Write and edit reviews	`reviews`, `ratings`, `albums`, `songs`, `artists`, `users`, `review_reports`	`reviews`, `ratings`, `artists`, `albums`, `songs`	Reused
3. Manage listening diary	`diary_entries`, `user_settings`, `albums`, `songs`, `artists`, `users`	`diary_entries`, `user_settings`, `artists`, `albums`, `songs`	`diary_entries.updated_at` column added

Use case	Tables read	Tables written	Sprint 3 change
4. Post comments on reviews	comments, reviews, review_reports, users	comments	Reused

Table 7.5: Key primary-key and foreign-key relationships

Table	Primary key	Important foreign keys and constraints
users	user_id	UNIQUE username, UNIQUE email
artists	artist_id	UNIQUE (external_source, external_id) – one row per MusicBrainz artist
albums	album_id	artist_id → artists (RESTRICT); UNIQUE (external_source, external_id)
songs	song_id	album_id → albums (SET NULL); artist_id → artists; UNIQUE (external_source, external_id)
reviews	review_id	user_id → users; album_id → albums; song_id → songs (all CASCADE). CHECK: exactly one of album/song. UNIQUE (user_id, album_id) and (user_id, song_id)
ratings	rating_id	Same links and uniqueness as reviews; CHECK score between 0.5 and 5.0
comments	comment_id	review_id → reviews (CASCADE); user_id → users (CASCADE)
diary_entries	diary_entry_id	user_id → users; album_id → albums; song_id → songs (CASCADE). CHECK: exactly one of album/song. No uniqueness, so repeat listens are allowed
user_settings	user_id	user_id → users (CASCADE); diary_visibility ENUM('public', 'private'), default public

Connecting MusicBrainz data to local records. Artists, albums and songs that users review or log are stored with `external_source = 'musicbrainz'` and `external_id` set to the MusicBrainz ID (MBID). The backend saves them with `INSERT ... ON DUPLICATE KEY UPDATE`, using the unique key on (`external_source`, `external_id`), so it either creates the row or reuses the existing one in a single statement. Two users reviewing the same album therefore receive the same `album_id`, and no duplicate albums are created even if they submit at the same moment. This is done inside a transaction together with the artist.

Associating content with users. Reviews, ratings, comments and diary entries all store the `user_id` of the logged-in user, taken from the session on the server. They are deleted automatically if the user account is deleted (ON DELETE CASCADE).

Persistence across sessions and computers. Because all data lives in RDS rather than in the browser or on one laptop, it remains after logging out, refreshing, or restarting the backend. Any team member who runs the backend with the same `.env` settings connects to the same database and sees the same reviews, comments and diary entries. Note that login sessions themselves are kept in the memory of each backend process, so a user logs in again on each computer or after a backend restart.

Inspecting records with MySQL Workbench. The team connects MySQL Workbench to the RDS endpoint to check that the application saved the expected data. The repository includes `database/verify_sprint3.sql`, a read-only script that lists the MusicBrainz-linked albums and songs, the reviews with their ratings, comments, diary entries, diary visibility settings, and checks that there are no duplicate albums or orphaned comments. Figure 7.16 is reserved for a Workbench screenshot of these results.

7.6 External API Integration

Why an external API. Storing a complete music catalog ourselves would mean importing and maintaining millions of artists, albums and tracks. Instead, TuneBoxd uses MusicBrainz, a free, community-maintained music encyclopedia, for music metadata, and only stores the albums, songs and artists that users actually review or log. MySQL remains the source of truth for everything users create. The public MusicBrainz API **does not require an API key**; it only asks that each application identify itself with a descriptive User-Agent and stay under one request per second.

What MusicBrainz provides. Search results and details for artists, albums and songs, including artist names, album titles, release dates, album types (album, EP, single), tracklists with track lengths, genres, and stable identifiers (MBIDs). The **Cover Art Archive** provides album cover images for the same MBIDs.

Table 7.6: How TuneBoxd concepts map to MusicBrainz and MySQL

TuneBoxd	MusicBrainz entity	MySQL table	Stored identifier
Artist	Artist	`artists`	artist MBID
Album	Release group (all editions of an album)	`albums`	release-group MBID
Song	Recording	`songs`	recording MBID

We map albums to MusicBrainz *release groups* rather than individual *releases*, because MusicBrainz stores each edition (CD, vinyl, remaster, country) as a separate release. Using the release group means all reviews of an album are collected on one page regardless of edition.

Searching. `GET /api/music/search` calls the MusicBrainz search endpoints for release groups, artists or recordings with `dismax=true`. In this mode MusicBrainz treats the query as

plain words matched against titles and artist names, so a search such as "ok computer radiohead" works, and user input is never interpreted as query syntax. Results are returned 20 per page.

Album and artist details. For an album, the backend looks up the release group (including artist credits, editions and genres), selects the earliest official release with an exact date, and looks up that release's tracklist. For an artist, it looks up the artist and then lists up to 100 of the artist's albums and EPs, studio albums first.

Artwork. For album pages, the backend requests the Cover Art Archive metadata for the release group and uses the front cover's image links. Search results use Cover Art Archive's direct front-cover image address, which the browser loads; if no cover exists, the frontend shows a placeholder instead of a broken image.

How Express communicates with the APIs. All MusicBrainz requests go through one backend module (`musicBrainzClient.js`) that uses Node's built-in `fetch`, sends the header `User-Agent: TuneBoxd/0.1.0 ( <contact> )` (the contact can be set with `MUSICBRAINZ_CONTACT` and defaults to our GitHub URL), and converts responses into the simplified objects that the frontend uses.

Table 7.7: Rate limiting, caching and error handling

Situation	How TuneBoxd handles it
MusicBrainz rate limit (1 request/second)	A shared queue starts at most one MusicBrainz request every 1.1 seconds, no matter how many users are searching. If more than 20 requests are waiting, new ones get "Music search is busy" (HTTP 503) instead of waiting a long time.
MusicBrainz returns 503 or 429 (busy)	The backend waits (2 seconds, then 4 seconds) and retries up to three attempts in total before returning a friendly 503 message.
Repeated requests	Responses are cached in memory (searches for 10 minutes, details for 6 hours, cover information for 24 hours, up to 500 entries). Identical requests that are already in progress share one call.
Invalid or unknown ID	IDs are checked against the MBID (UUID) format first (400). An unknown ID returns 404, e.g. "We couldn't find that album."
Network failure or timeout	Requests time out after 10 seconds and return "MusicBrainz is temporarily unavailable" (503). If MusicBrainz is down but the album already exists in MySQL, reviews and diary entries can still be saved using the existing row.
Missing artwork or incomplete metadata	Missing covers show a placeholder; missing tracklists show a message. Partial dates such as "2007" are displayed, but stored as NULL in the DATE column.

7.7 How to Build and Run TuneBoxd

TuneBoxd can be built and run entirely from a regular terminal (Command Prompt, PowerShell or Git Bash on Windows; Terminal on macOS or Linux). No IDE is required. The commands below were tested on Windows 11 with Node.js 22 and npm 10.

Step 1: Install the prerequisites

- **Node.js 20.12 or newer**, which includes **npm**: download the LTS installer from nodejs.org. The backend uses `process.loadEnvFile`, which requires Node.js 20.12 or later.
- **Git**: download from git-scm.com.
- Network access to our AWS RDS database (see Step 4).

Check the installations:

```
node --version
npm --version
git --version
```

Step 2: Clone the repository and open the project folder

```
git clone https://github.com/Vanohra/TuneBoxd.git
cd TuneBoxd
```

Step 3: Install the dependencies

The backend and frontend each have their own `package.json`. From the repository root, one command installs both:

```
npm run setup
```

This is the same as running `npm install` inside `backend` and then inside `frontend`.

Step 4: Configure the environment variables

Copy the example file to `.env` in the repository root and fill in the database values (ask the team for the RDS credentials; never commit `.env` or share it publicly):

```
copy .env.example .env      (Windows Command Prompt)
cp .env.example .env        (macOS, Linux or Git Bash)
```

Variable	Required	Description
DB_HOST	Yes	RDS endpoint hostname
DB_PORT	No	MySQL port (default 3306)
DB_USER	Yes	Database user name
DB_PASSWORD	Yes	Database password
DB_NAME	No	Database name (default music_review_app)
SESSION_SECRET	Production only	Long random string used to sign the session cookie
MUSICBRAINZ_CONTACT	No	Email or URL sent in the MusicBrainz User-Agent
PORT	No	Backend port (default 4000)

RDS network access. The computer running the backend must be able to reach the RDS instance on port 3306. The RDS instance must be publicly accessible, and its AWS security group must have an inbound rule that allows MySQL/Aurora (TCP 3306) from that computer's public IP address. If this is missing, the backend stops at startup with "Unable to connect to database (ETIMEDOUT)".

If the database has not been migrated yet, apply the Sprint 3 migration once (it is safe to run again):

```
npm run migrate
```

Step 5: Start the application

Option A – one terminal (from the repository root):

```
npm run dev
```

This starts both servers. The terminal should show "Database connection successful.", "TuneBoxd API listening on http://localhost:4000" and Vite's "Local: http://localhost:5173/".

Option B – two terminals:

```
Terminal 1 (backend):
cd TuneBoxd/backend
npm run dev

Terminal 2 (frontend):
cd TuneBoxd/frontend
npm run dev
```

Step 6: Open the application

Open a web browser and go to the address shown by Vite, **http://localhost:5173**. Keep the backend running, because the frontend sends all `/api` requests to it.

Step 7: Stop the application

Press **Ctrl + C** in each terminal that is running a server (answer "Y" if Windows asks "Terminate batch job?").

Building for production

The frontend can be compiled into optimized static files:

```
npm run build        (from the root; same as "npm run build" in frontend)
npm run preview      (serves the built files on http://localhost:4173)
```

The build is written to `frontend/dist`. The preview server forwards `/api` requests to the backend in the same way as the development server, so the backend must be running. The backend does not need a build step; it can be started without file watching using `npm run start:backend` from the root (or `npm start` in `backend`). The automated tests can be run with `npm test` from the root.

7.8 Implementation Testing and Verification

We verified the implementation in three ways: an automated test suite, a walkthrough of the user interface in a browser, and database queries.

Automated tests. The backend includes unit tests (`backend/test/unit.test.js`) for validation rules and MusicBrainz data mapping, and end-to-end API tests (`backend/test/api.test.js`) that start the real Express app and send HTTP requests against **our actual RDS database and the live MusicBrainz API**. The API tests register temporary users, exercise all four use cases and the Sprint 2 features, check the saved rows directly in MySQL, and delete the temporary users at the end. On October 18, 2026, `npm test` ran **55 tests and all 55 passed**.

Interface walkthrough. The complete flow shown in Section 7.3 (register, log in, search, open an album, write a review, add diary entries, log out, and comment from a second account) was carried out in a browser against the running application. Editing and deleting a diary entry, switching the diary to private, viewing the private diary as a logged-out visitor, and the layout at phone width (375 px) were also checked in the browser.

Table 7.8: Verification results

What was verified	How	Result
Music search returns real MusicBrainz data (albums, artists, songs); empty and no-match searches handled	API tests; browser (Figure 7.6)	Passed
Album details: title, artist, release date, 10-track tracklist, artwork; invalid and unknown IDs give 400/404	API tests; browser (Figure 7.7)	Passed
Writing a review: saved to `reviews` and `ratings`, linked to the user and the album, visible after a new request	API tests (checked rows in MySQL); browser (Figure 7.9)	Passed
Two users reviewing one album share one `albums` row (no duplicates)	API test	Passed
Editing a review: text and rating change, `created_at` kept, `updated_at` updated	API test	Passed (edit form not separately shown in the screenshots)
Review validation: empty text, invalid rating, duplicate review, unknown album rejected	API tests; browser	Passed
Adding diary entries (including the same album twice), editing and deleting them	API tests; browser (Figures 7.10–7.11)	Passed
Private diaries hidden from other users and visitors (403)	API test; browser	Passed
Posting, editing and deleting comments; persistence after refresh	API tests; browser (Figure 7.12)	Passed

What was verified	How	Result
Removed (moderated) reviews hidden and cannot be commented on; deleting a review leaves no orphaned comments	API tests	Passed
Authentication and ownership: guests get 401; editing or deleting another user's review, comment or diary entry gets 403	API tests; browser	Passed
Sprint 2 regression: session persistence, protected routes, profile editing, following, duplicate registration, wrong password, logout	API tests; browser (Figures 7.13–7.14)	Passed
Stored records checked with SQL (`database/verify_sprint3.sql`)	Script executed against RDS through the backend's MySQL connection	Passed
Stored records inspected in MySQL Workbench	Team member with Workbench access	Not yet performed (screenshot needed, Figure 7.16)
Another user on a different computer retrieves the same reviews and comments	Two computers running the app against RDS	Not yet performed
Running from a terminal: `npm run setup`, `npm run dev` (both servers), `npm run build`, `npm run preview`	Terminal on Windows 11	Passed

Remaining tests: the cross-computer check should be done by having one team member post a review and another member, on a different computer, open the same album page and comment on it. The Workbench screenshot should then show both records.